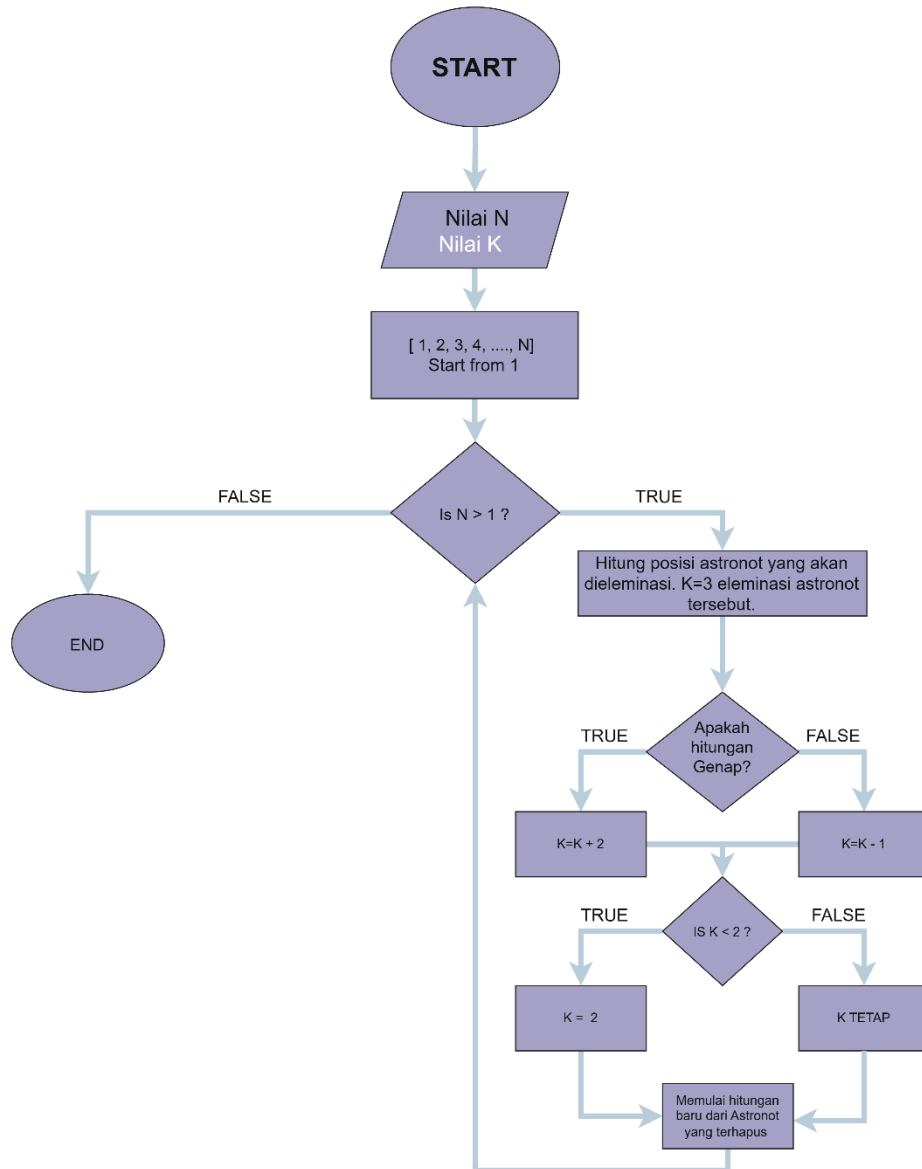


The Last Astronout

Flowchart Algorithm:



Penjelasan algoritmanya: Algoritma ini bekerja dengan cara kita meng-input nilai **N** dan **K** yang dimana **N** adalah sebuah bilangan bulat positif dan **K** adalah 3, karena harus lebih besar dari 2, algoritma ini mengeleminasi astronout pada hitungan **K** nya sampai dengan astronoutnya tersisa 1, dan jika hitungan **K** nya adalah genap maka **K**-nya akan di tambah dua, jika **K**-nya bernilai ganjil maka **K** akan dikurangkan dengan satu, dan jika dari hasil pengurangan tersebut menghasilkan $K < 2$ maka nilai **K**-nya akan menjadi 2, jika tidak maka nilai **K**-nya tetap. Proses ini akan di ulangi hingga astronot berjumlah satu.

Alur Algoritma "The Last Astronaut"

1. **START** — mulai program.
2. **Input** nilai **N** (jumlah astronot) dan nilai **K** (nilai hitungan awal untuk eliminasi).
3. **Inisialisasi lingkaran astronot**: dibuat array/daftar astronot bernomor $[1, 2, 3, \dots, N]$, dan penghitungan dimulai dari astronot nomor 1.
4. **Pengecekan kondisi berhenti** — Is $N > 1$?
 - Jika **FALSE** (N sudah = 1, artinya hanya tersisa satu astronot) → langsung menuju **END**.
 - Jika **TRUE** (masih ada lebih dari satu astronot) → lanjut ke proses eliminasi berikut.
5. **Hitung posisi astronot yang akan dieliminasi**: dari posisi awal hitungan, hitung maju sebanyak K langkah, lalu astronot pada hitungan ke- K tersebut dieliminasi dari lingkaran.
6. **Cek paritas nomor astronot yang baru dieliminasi** — Apakah hitungan Genap?
 - **TRUE** (nomor astronot genap) → $K = K + 2$
 - **FALSE** (nomor astronot ganjil) → $K = K - 1$
7. **Validasi nilai K minimum** — Is $K < 2$?
 - **TRUE** (K jadi kurang dari 2 setelah perubahan) → K dipaksa menjadi 2
 - **FALSE** (K masih ≥ 2) → K TETAP (nilai K hasil langkah 6 dipakai apa adanya)
8. **Mulai hitungan baru**: proses penghitungan berikutnya dimulai dari posisi tepat setelah astronot yang baru saja dieliminasi.
9. **Kembali (loop)** ke langkah pengecekan Is $N > 1$? — proses ini berulang terus sampai jumlah astronot yang tersisa di lingkaran hanya satu.
10. **END** — program berhenti ketika hanya tersisa satu astronot, yaitu astronot terakhir yang bertahan (jawaban dari soal).

Alien In The Middle

Pseudocode:

```
#include <iostream>

#include <string>

using namespace std;

// Mengubah huruf menjadi nilai (A=1, B=2, ..., Z=26)
int hurufKeNilai(char c) {
    return (c - 'A') + 1;
}

// Mengubah nilai menjadi huruf, dengan perhitungan ulang jika > 26
char nilaiKeHuruf(int v) {
    // Jika melebihi 26, kembali berputar dari A
    v = ((v - 1) % 26) + 1;
    return 'A' + (v - 1);
}

// Fungsi untuk mengenkripsi pesan
string enkripsiPesan(string pesanAsli) {
    string pesanSandi = pesanAsli; // salin ukuran string
    int n = pesanAsli.length();

    // Huruf pertama tidak mengalami perubahan
    pesanSandi[0] = pesanAsli[0];

    // Mulai dari huruf kedua
    for (int i = 1; i < n; i++) {
        int nilaiSekarang = hurufKeNilai(pesanAsli[i]);
        int nilaiSebelumnya = hurufKeNilai(pesanAsli[i - 1]);

        int hasil = nilaiSekarang + nilaiSebelumnya;
```

```

        // Jika hasil melebihi 26, kurangi 26 (perhitungan kembali dari A)
        if (hasil > 26) {
            hasil -= 26;
        }
        pesanSandi[i] = nilaiKeHuruf(hasil);
    }

    return pesanSandi;
}

int main() {
    string pesan;
    cout << "Masukkan pesan asli (huruf kapital tanpa spasi): ";
    cin >> pesan;

    string hasilSandi = enkripsiPesan(pesan);

    cout << "\nProses penyandian:\n";
    cout << "Pesan Asli : " << pesan << endl;
    cout << "Pesan Sandi : " << hasilSandi << endl;

    return 0;
}

```

Cara Kerjanya:

Langkah-Langkah Algoritma

1. Fungsi hurufKeNilai(char c)

Mengonversi huruf menjadi angka posisinya di alfabet, dengan A=1, B=2, ..., Z=26.

2. Fungsi nilaiKeHuruf(int v)

Kebalikannya, mengonversi angka kembali menjadi huruf. Baris $v = ((v - 1) \% 26) + 1$; memastikan jika nilai v melebihi 26, angka tersebut akan “berputar” kembali ke rentang 1–26 (misalnya 27 menjadi 1 fi A, 28 menjadi 2 fi B, dst). Fungsi ini sebenarnya jaring pengaman tambahan, karena enkripsiPesan sudah menangani pengurangan 26 secara manual sebelumnya.

3. Fungsi enkripsiPesan(string pesanAsli) – Inti Penyandian

- Huruf pertama (indeks 0) tidak diubah – langsung disalin apa adanya ke pesan sandi.
- Untuk setiap huruf berikutnya (indeks ke-1 dan seterusnya):

l Mengambil nilai huruf saat ini (nilaiSekarang) dan nilai huruf sebelumnya dari pesan asli (nilaiSebelumnya) – penting: yang dipakai adalah huruf asli, bukan huruf yang sudah disandikan.

l Menjumlahkan keduanya: hasil = nilaiSekarang + nilaiSebelumnya.

l Jika hasil penjumlahan melebihi 26, dikurangi 26 supaya kembali ke rentang 1–26 (logika modulo

manual, seperti jam yang berputar dari Z kembali ke A).

l Hasil akhir diubah kembali menjadi huruf lewat nilaiKeHuruf(), dan menjadi huruf sandi pada

posisi tersebut.

4. Fungsi main()

Meminta input pesan asli (huruf kapital tanpa spasi), memanggil enkripsiPesan(), lalu menampilkan pesan asli dan pesan sandi hasil enkripsi